

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE Cloud Computing and
NAME: Big Data Analytics

COURSE
CODE: CSA1501

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.

(BL5)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Articulation (Level 4)

Total Marks: 50

Question Count: 5

Code of Conduct

1. _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts correctly	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. HDFS-Based Storage Solution for a Media Platform

Proposed Design

- Use an HDFS cluster with two NameNodes in active-standby high-availability mode, three JournalNodes, and multiple DataNodes distributed across different racks.
- Set the default replication factor to 3 for critical media metadata, newly uploaded files, and frequently accessed content. Use replication factor 2 only for low-risk temporary or re-creatable data so that the maximum never exceeds 3.
- Enable rack awareness so that replicas are placed across different racks. This protects data from both disk and rack failures and supports the required availability target.
- Use HDFS erasure coding for cold or archival media where appropriate. It gives fault tolerance with much lower storage overhead than maintaining three full replicas.
- Apply HDFS storage policies: SSD/HOT for popular metadata and indexes, DISK/WARM for active media, and ARCHIVE for older content.

Availability Above 99.9%

NameNode HA removes the NameNode single point of failure. DataNode replication, rack-aware placement, automatic block re-replication, and health monitoring allow service to continue after individual node or disk failures. JournalNodes keep namespace changes synchronized so that the standby NameNode can take over quickly.

Cost-Efficient Growth

Add commodity DataNodes horizontally as capacity increases, compress suitable metadata, move inactive files to archival tiers, and use erasure coding for cold data. HDFS Balancer redistributes blocks when new nodes are added, preventing capacity hotspots.

Justification

This design satisfies all constraints: replication never exceeds 3, high availability protects service continuity, and tiered storage plus erasure coding reduce the cost of large media growth without sacrificing required durability.

Operational Management and Monitoring

Use HDFS fsck, NameNode health metrics, DataNode heartbeats, and centralized monitoring such as Ambari/Prometheus/Grafana to detect missing blocks, disk pressure, and failed nodes early. Automated alerts allow administrators to replace failing disks or nodes before availability drops below the required 99.9% threshold.

Capacity planning should reserve free space on every DataNode and add nodes before utilization becomes critical. HDFS Balancer can redistribute blocks after expansion, while quota policies prevent a single content category from consuming uncontrolled storage.

For frequently streamed media, metadata and thumbnails can be separated from very large video objects. Small metadata files can be consolidated using sequence/container formats to reduce NameNode memory pressure, while large media files remain efficient HDFS blocks.

Failure Scenario

If one DataNode fails, clients continue reading another replica. If an entire rack fails, rack-aware placement preserves at least one copy on another rack. If the active NameNode fails, automatic failover promotes the standby NameNode, limiting service interruption and supporting the availability requirement.

Final Design Decision

The platform therefore combines replication factor 3 for critical active data, replication factor 2 or erasure coding for recoverable/cold data, NameNode HA, rack awareness, snapshots, monitoring, and horizontal scaling. This achieves high availability without exceeding the replication constraint and keeps long-term storage growth economical.

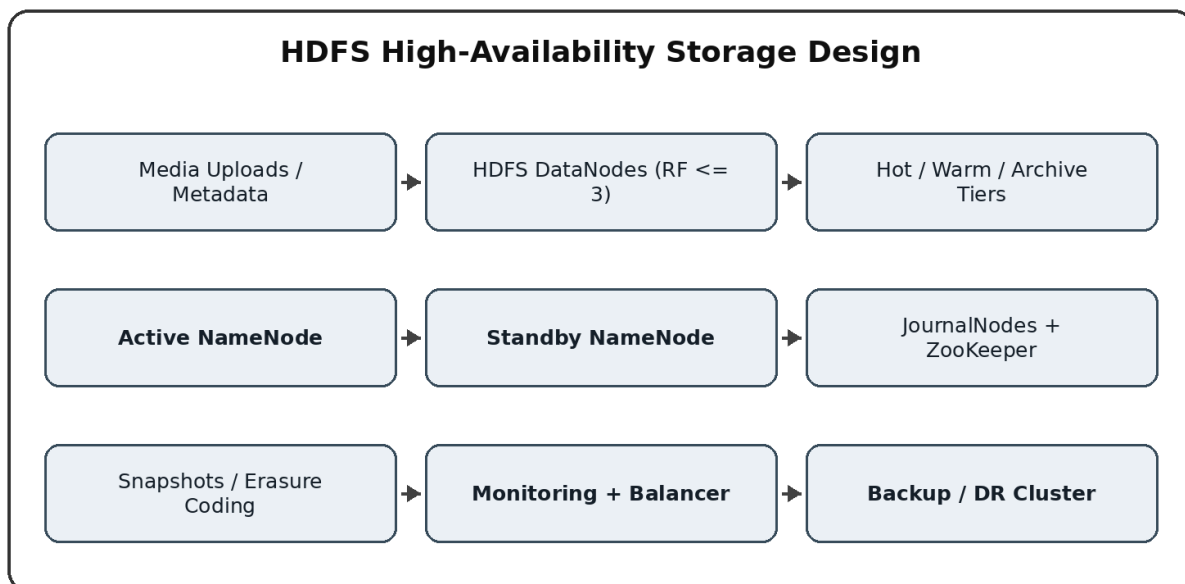


Figure 1: Suggested architecture for Question 1

2. MapReduce Workflow for a 20-Minute Batch Window

Job Design

- Store logs in HDFS as time-partitioned files such as /logs/yyyy/mm/dd/hh and process only the new batch for each execution.
- Combine very small log files before processing because excessive small files create too many map tasks and NameNode metadata operations.
- Use splittable compressed formats such as SequenceFile, Avro, or Parquet where possible. Input splits should be close to the HDFS block size so each mapper receives a useful amount of work.
- Perform filtering, parsing, and partial aggregation in map tasks. Use a Combiner for associative operations such as counts and sums to reduce shuffle traffic.
- Use a custom Partitioner when keys are skewed so that hot keys do not overload a single reducer. Configure only as many reducers as the limited cluster can run efficiently.

Task Distribution

YARN launches map tasks close to the DataNodes that hold input blocks, improving data locality and reducing network transfer. Maps execute in parallel. Their intermediate outputs are partitioned by key, shuffled to reducers, sorted, and then aggregated. Reducer count is tuned to available vCores and memory instead of creating excessive parallel tasks.

Meeting the 20-Minute Constraint

Use incremental batches, map-side filtering, combiners, compression of shuffle output, speculative execution for unusually slow tasks, and appropriate mapper/reducer memory settings. Avoid unnecessary joins; when a small reference table is required, use DistributedCache/map-side join so the workflow completes in a single major MapReduce stage.

Expected Result

The workflow maximizes useful parallelism while avoiding task-startup and shuffle overhead, making the 20-minute target realistic even with limited nodes.

Detailed Task Distribution

Suppose the cluster has only a small number of worker nodes. The job should create enough map tasks to keep all CPU cores busy, but not thousands of tiny tasks. Each mapper reads one or more balanced input splits, parses only required fields, rejects invalid records, and emits compact intermediate key-value pairs.

Reducers should be sized according to the available reducer slots or YARN containers. A small set of balanced reducers is better than many reducers that spend most of the

20-minute window starting containers and exchanging shuffle data. Skewed keys should be salted or custom-partitioned so that no single reducer becomes a bottleneck.

Performance Controls

Enable map-output compression, reuse JVM/container resources where practical, tune sort buffers, and avoid writing unnecessary intermediate files. Run the job on data stored locally in HDFS so YARN can schedule tasks with node or rack locality. Speculative execution can be enabled selectively for genuine stragglers.

The workflow should be benchmarked using sample batches. If the first stage consumes too much of the 20-minute budget, pre-filter logs during ingestion or use hourly partitions so each MapReduce run processes only newly arrived data rather than rescanning historical logs.

Expected Timing Outcome

With balanced input splits, mapper-side aggregation, controlled reducer count, compressed shuffle, and locality-aware scheduling, most of the batch time is spent on useful parallel processing rather than network movement or task overhead. This is the most suitable design when cluster nodes are limited.

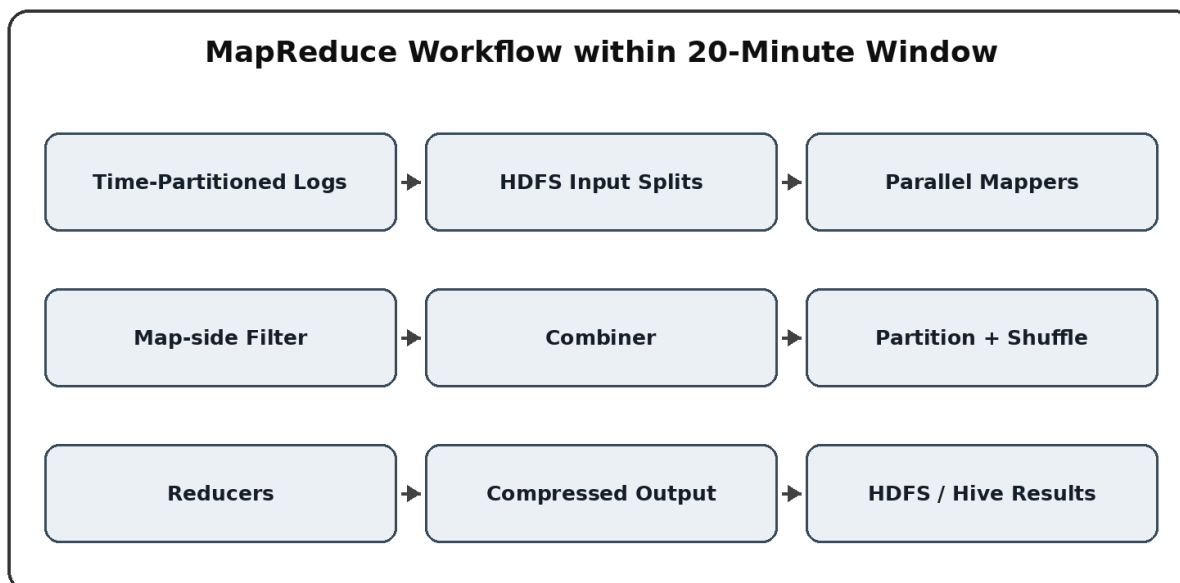


Figure 2: Suggested architecture for Question 2

3. YARN Scheduler Policy for ETL, Reporting, and Machine Learning

Recommended Scheduler

Use the YARN Capacity Scheduler with three queues: etl, reporting, and ml. Give ETL the highest guaranteed capacity and priority because it prepares data required by downstream jobs. Reporting and ML receive guaranteed minimum capacity, while unused capacity can be borrowed by other queues.

Example Policy

- ETL queue: 50% guaranteed capacity, high priority, maximum capacity up to 80%.
- Reporting queue: 25% guaranteed capacity, medium priority, with user limits to preserve fairness.
- ML queue: 25% guaranteed capacity, normal priority, with limits on very large training applications.
- Enable preemption so that when ETL needs its guaranteed resources, lower-priority borrowed containers can be reclaimed gradually.
- Apply queue ACLs, maximum application limits, and per-user capacity limits so one team cannot monopolize the cluster.

Fairness

Within each queue, multiple users share resources using user limits and application ordering. Idle ETL resources can temporarily be used by reporting or ML, maintaining high cluster utilization without losing ETL priority.

Recovery from Node Failure

YARN ResourceManager HA provides active-standby recovery for the cluster manager. If a NodeManager fails, running containers are marked lost and ApplicationMasters request replacement containers on healthy nodes. HDFS replication keeps input data available, while checkpointing in long-running ML jobs prevents complete restart.

Justification

The policy balances fairness, business priority, and fault recovery while keeping the cluster well utilized. Capacity guarantees protect ETL, and elastic borrowing avoids wasting idle resources.

Scheduler Rules and Guarantees

The ETL queue should receive a guaranteed minimum capacity because reporting and ML depend on fresh curated data. However, unused ETL capacity should not remain idle; it can be temporarily borrowed by reporting and ML jobs. When ETL demand increases, preemption gradually returns resources to the ETL queue.

Set maximum application limits and per-user limits in all queues. Reporting jobs can be assigned medium priority with moderate containers, while long-running ML training jobs can be capped so they do not consume all memory or GPUs/CPU cores. Small interactive reports may be prioritized inside the reporting queue.

Failure Recovery Strategy

ResourceManager HA should use active and standby ResourceManagers with ZooKeeper-based coordination. NodeManagers continuously send heartbeats. When a worker disappears, YARN marks its containers lost and the ApplicationMaster reschedules failed tasks on healthy nodes. MapReduce and Spark retry mechanisms recover work without restarting the entire workflow.

Application state and job history should be preserved using YARN recovery services and the JobHistory Server. Logs should be aggregated to HDFS so administrators can diagnose failures even when the original worker node is unavailable.

Result

This scheduler design satisfies all three constraints simultaneously: fairness is provided through queue/user limits, ETL receives guaranteed priority, and ResourceManager/NodeManager recovery mechanisms maintain execution when nodes fail.

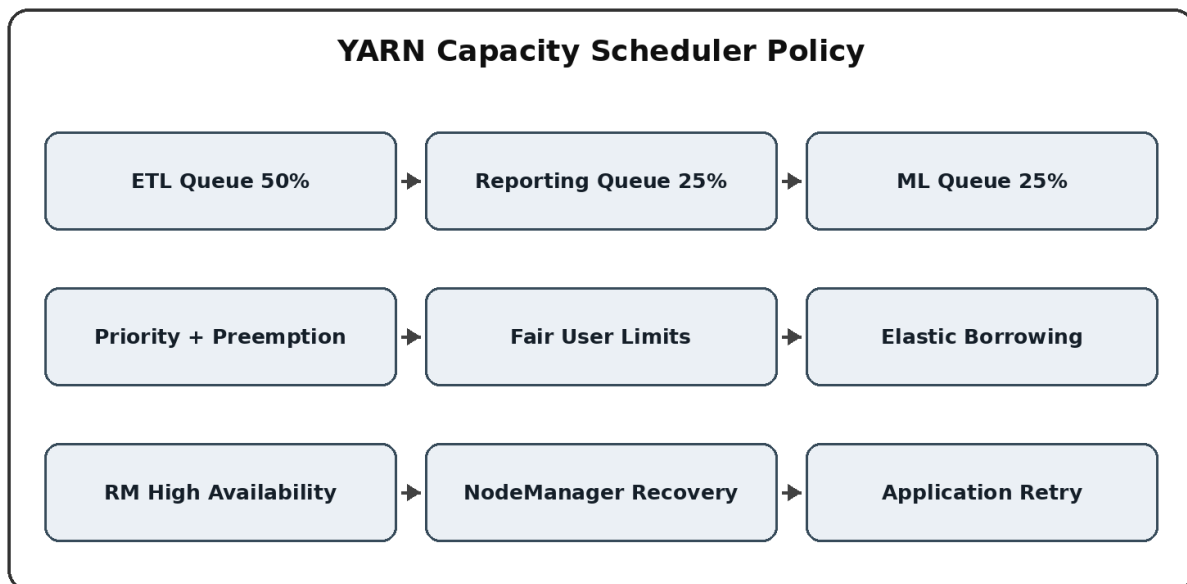


Figure 3: Suggested architecture for Question 3

4. Enterprise Ingestion from SAN, NAS, and Operational Databases

Constrained Architecture

- SAN/NAS file sources -> Apache NiFi or Hadoop DistCp -> HDFS landing zone.
- Operational relational databases -> Apache Sqoop or NiFi database processors using incremental/CDC-style extraction -> HDFS/Hive.
- HDFS raw zone stores the original source data. Curated zones store validated, transformed, and deduplicated datasets for analytics.
- Hive Metastore provides shared table definitions. Apache Spark or MapReduce performs cleansing, merging, and duplicate detection.
- Backup uses HDFS snapshots plus scheduled DistCp to a secondary HDFS cluster or object storage target.

Minimal Duplication

Use incremental ingestion based on timestamps, primary keys, log sequence numbers, or file checksums instead of copying complete sources repeatedly. NiFi maintains state and provenance, and file/database identifiers are recorded in a control table so already processed records are not reloaded.

Backup Reliability

HDFS replication protects against local node failures, while snapshots provide point-in-time recovery from accidental changes. DistCp copies changed blocks/files to a separate recovery location, protecting against site-level failure. Backup verification can compare counts, checksums, and control-table watermarks.

Component Justification

NiFi provides controlled multi-source ingestion and provenance; Sqoop is efficient for bulk/incremental RDBMS transfer; HDFS offers scalable replicated storage; Hive exposes structured data to SQL tools; Spark/MapReduce handles transformation and deduplication; snapshots and DistCp provide reliable disaster recovery without unnecessary full copies.

Source-Specific Ingestion

SAN and NAS systems normally expose large files and directories. NiFi can watch approved folders, validate file names and checksums, and move only new or modified files into HDFS. For large Hadoop-to-Hadoop copies, DistCp provides parallel transfer and can update only changed files.

Operational databases should use incremental extraction rather than daily full-table copies. Sqoop can import rows using an increasing ID or timestamp column, while NiFi JDBC processors or CDC tools can capture inserts and updates continuously. This reduces source-system load and avoids duplicate storage.

Data Quality and Lineage

Every incoming dataset should receive source, arrival time, schema version, checksum, and batch identifier metadata. NiFi provenance records provide a trace of where each file came from and which processors changed it. Spark/Hive jobs can then perform schema validation, standardization, record matching, and deduplication before publishing curated tables.

Backup should be independent of local HDFS replication. HDFS snapshots protect against accidental deletion or corruption, while DistCp to a secondary cluster or object storage creates a disaster-recovery copy. Retention rules should keep multiple restore points without retaining unnecessary duplicate raw files forever.

Architecture Benefit

The resulting architecture delivers reliable multi-source ingestion, controlled incremental movement, data lineage, minimal duplication, and recoverable backups. It also separates the raw landing area from curated analytics data, improving governance and troubleshooting.

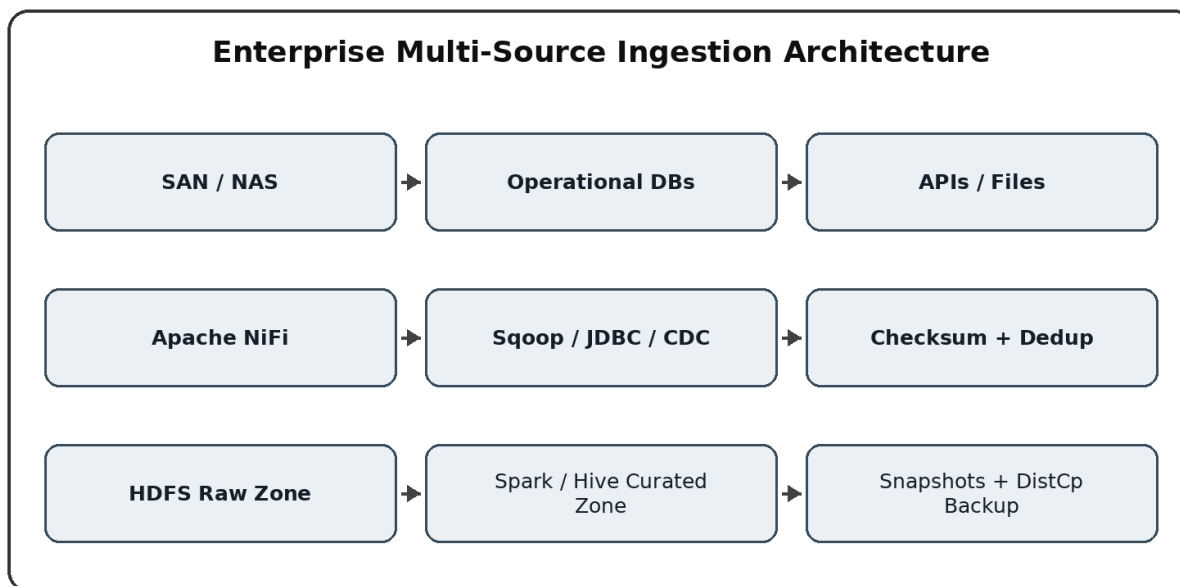


Figure 4: Suggested architecture for Question 4

5. End-to-End Secure and Highly Available Hadoop Ecosystem

Architecture Flow

- Data Sources -> Apache NiFi / ETL -> Kerberos-authenticated HDFS landing zone -> Hive/Spark/MapReduce processing -> curated Hive/HBase data -> BI/analytics consumers.
- NiFi runs in a clustered configuration and uses secure site-to-site or HTTPS/TLS connections. Sensitive attributes and credentials are stored using protected configuration and enterprise secret management.
- HDFS uses active-standby NameNodes, three JournalNodes, ZooKeeper Failover Controller, rack-aware DataNodes, and replication up to 3.
- YARN uses ResourceManager HA and multiple NodeManagers. Spark/MapReduce jobs execute on YARN for distributed processing.
- Hive Metastore and related services use redundant instances and backed-up metadata databases.

Secure Data Movement

Kerberos provides strong service and user authentication. TLS encrypts NiFi, Hadoop RPC/web endpoints, and inter-service traffic where configured. HDFS encryption zones protect data at rest. Apache Ranger can centrally enforce HDFS, Hive, HBase, and Kafka authorization policies and generate audit records.

High Availability

NameNode HA, ResourceManager HA, redundant NiFi nodes, replicated DataNodes, ZooKeeper coordination, and metadata database backups remove major single points of failure. Failed containers are restarted by YARN, and HDFS automatically restores under-replicated blocks.

NiFi / ETL Integration

NiFi processors can ingest files, APIs, queues, and databases; validate schema; route data; enrich metadata; and write to HDFS, Hive, Kafka, or downstream ETL stages. Back pressure prevents downstream overload, while data provenance provides end-to-end traceability.

How the Constraints Are Satisfied

Secure movement is achieved through Kerberos, TLS, encryption zones, and Ranger policies. High availability is achieved through active-standby masters and replicated workers. NiFi and ETL tools provide flexible, traceable integration with the Hadoop ecosystem while YARN and Spark/MapReduce provide scalable processing.

End-to-End Processing Stages

At the ingestion layer, NiFi receives files, API data, queue events, and database records. Process groups validate schema, remove duplicates, attach metadata, and route sensitive data through secure connections. Back-pressure and guaranteed delivery protect the platform when downstream Hadoop services are temporarily slow.

At the storage and processing layer, HDFS stores raw and curated data, Hive provides SQL metadata, HBase can support low-latency key-based access, and Spark/MapReduce jobs execute through YARN. Data can move from raw to cleansed, trusted, and consumption zones after validation and policy checks.

Security and Governance

Kerberos authenticates users and services. TLS protects network traffic. HDFS encryption zones protect sensitive files at rest, and Apache Ranger defines fine-grained policies for HDFS, Hive, HBase, and related services. Centralized audit logs record who accessed which data and when.

High availability should cover every critical master service: active-standby HDFS NameNodes, ResourceManager HA, clustered NiFi nodes, redundant metadata databases, ZooKeeper quorum, and multiple DataNodes across racks. Regular snapshots and off-cluster backups provide disaster recovery beyond node-level redundancy.

Constraint Verification

Secure data movement is met by authenticated and encrypted NiFi/Hadoop communication; high availability is met by redundant masters, replicated workers, automatic failover, and recovery; integration is met by NiFi processors and ETL jobs that can write directly to HDFS, Hive, Kafka, and downstream processing stages. The architecture is therefore secure, scalable, recoverable, and suitable for enterprise Hadoop workloads.

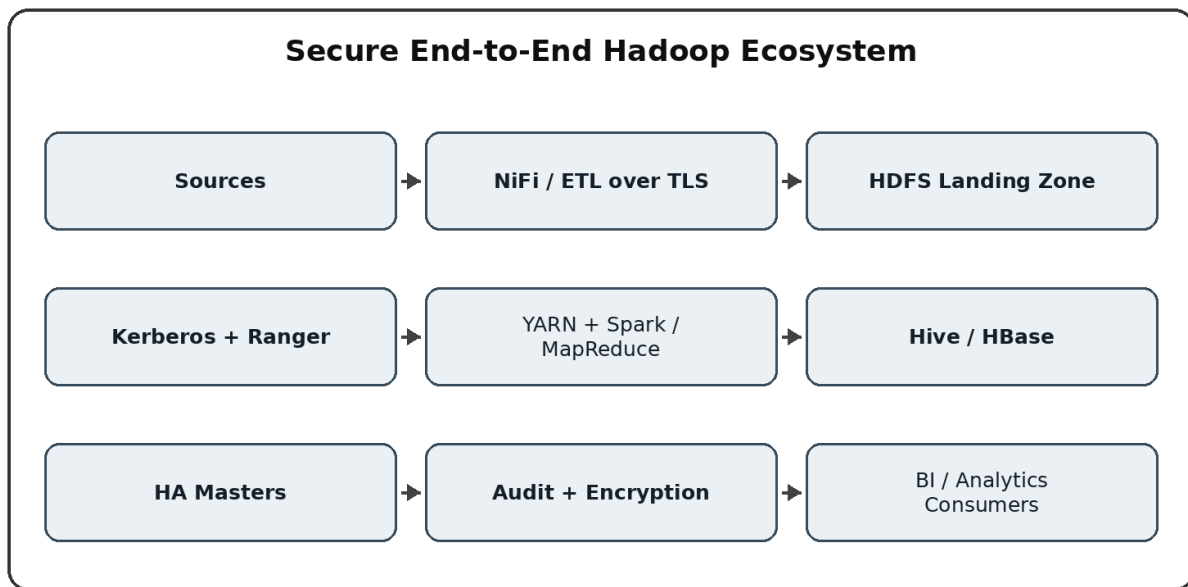


Figure 5: Suggested architecture for Question 5